

Bolco de estruturação/Descodificação V1.3

Data: 8/01/2026

Autor: Luís Costa

Índices

1.	Introdução	2
2.	Diagrama do Bloco de Memória (16 Bytes)	2
3.	Implementação em C/C++ (Struct)	4
4.	Funções de Codificação (Packing)	4
5.	Implementação da Encriptação (AES-128)	5
4.1.	Chave de Segurança	5
4.2.	Funções de Cifra e Decifra	5
6.	Código do Emissor atualizado (Colete)	6
7.	Código do Recetor atualizado (Tablet / Dashboard)	11
8.	Protocolo de Sincronização e Controlo (TDMA)	15
	Lógica de Receção no Firmware	18
	Mudanças no código do emissor atualizado	18
	Mudanças no código do recetor atualizado	19
9.	Conformidade Regulamentar (Banda ISM)	19
	Análise de Tempo de Antena (Time on Air - ToA)	20
	Cálculo de Ocupação (Duty Cycle)	20
10.	Notas finais a ter em conta para os proximos passos	21
11.	Bibliografia	21
12.	Historico de versões	22

1. Introdução

Este documento detalha a estrutura de dados, o protocolo de segurança e as regras de transmissão via LoRa para o projeto Wearion. O sistema foi desenhado para garantir eficiência energética, segurança (AES-128) e conformidade legal (ETSI EN 300 220), utilizando um bloco de memória de dados fixo de 16 bytes.

2. Diagrama do Bloco de Memória (16 Bytes)

A tabela que se segue representa como os bits estão organizados na memória antes da encriptação AES-128.

		Bits								Descrição
		7	6	5	4	3	2	1	0	
Bytes	00	Vest ID (int8_t)								ID único
	01	FLG HW	Man Down	High BPM	Low BPM	High Temp	Low Temp	Low Battery	FLG SOS	Flags Alerta
	02	Heart Rate (8 bits)								HR (9 bits)/(8 bits)? + SpO2 (7 bits)
	03	SpO2 (7 bits)							HR (1bit)	
	04	Temperature (int16_t)								Temp * 100
	05									
	06 ... 09	Latitude (int32_t)								Lat *10^7

	10	Longitude (int32_t)	Lon *10 ⁷
	...		
	13		
	14	Altitude (int16_t)	Altitude em Metros
	15		

Descrição das flags

FLG HW (Hardware Fault) → ativa se houver alguma falha de hardware num específico colete.

Man Down (Queda/Inatividade) → ativa se o operacional cair ou estiver com uma posição anormal.

High BPM (Taquicardia) → ativa quando o sensor dos BPM recebe bpms acima do esperado para o bem estar do operacional (ex: >160 BPM).

Nota: Estimativa da Frequência Cardíaca Máxima em esforço (FC Máx) : A forma mais comum de estimar o limite máximo é a fórmula → 220 - a sua idade.

Low BPM (Bradycardia) → ativa quando o sensor dos BPM recebe bpms abaixo do esperado para o bem estar do operacional (ex: <45 BPM).

High Temp (Hipertermia) → ativa quando o sensor da temperatura corporal recebe valores acima do esperado para o bem estar do operacional, essencial no combate a incêndios (ex: >40°).

Low Temp (Hipotermia) → ativa quando o sensor da temperatura corporal recebe valores abaixo do esperado para o bem estar do operacional (ex: <35°).

Low Battery → ativa quando a bateria do colete estiver a $X\%$, alertando o capitão para, quando possível, substituir a bateria.

FLG SOS (Botão de SOS integrado no colete) → Flag de prioridade máxima ativado manualmente pelo operacional através do botão SOS em caso de perigo iminente.

3. Implementação em C/C++ (Struct)

Para garantir que o compilador não adiciona "padding" (espaços vazios) entre as variáveis, é obrigatório o uso do atributo packed.

```
typedef struct __attribute__((packed)) {  
    int8_t vestID;  
    uint8_t spo2_header;  
    uint16_t heartRate;  
    int16_t temperature;  
    int32_t latitude;  
    int32_t longitude;  
    int16_t altitude;  
} ResQSensePacket;
```

4. Funções de Codificação (Packing)

Como os campos spo2_header e heartRate misturam dados e flags no mesmo byte, não se deve escrever neles diretamente. Utilizamos funções auxiliares para garantir que os bits ficam na posição correta.

Notas Importantes de Conversão:

Temperatura: 36.50°C deve ser enviado como 3650.

Coordenadas: 40.6412° deve ser enviado como 406412000.

Heart Rate: Suporta até 511 BPM (9 bits), cobrindo casos de taquicardia extrema.

```
ResQSensePacket createPacket(int id, bool alertFlag, int spo2, int hr, float temp, float lat, float lon, int alt) {  
    ResQSensePacket packet;  
  
    packet.vestID = (int8_t)id;  
    uint8_t flagBit = alertFlag ? 1 : 0;  
    packet.spo2_header = (flagBit << 7) | (spo2 & 0x7F);  
    packet.heartRate = (uint16_t)(hr & 0x01FF);  
    packet.temperature = (int16_t)(temp * 100.0f);  
    packet.latitude = (int32_t)(lat * 10000000.0f);  
    packet.longitude = (int32_t)(lon * 10000000.0f);  
}
```

```

    packet.altitude = (int16_t)alt;

    return packet;
}

```

5. Implementação da Encriptação (AES-128)

Para garantir a confidencialidade dos dados transmitidos via LoRa, utiliza-se o algoritmo **AES-128**. Este modo foi escolhido por permitir encriptar o bloco de 16 bytes da estrutura ResQSensePacket num bloco cifrado de exatamente 16 bytes, sem aumentar o tamanho do payload e mantendo a eficiência energética.

O ESP32 possui aceleração de hardware para criptografia através da biblioteca nativa mbdts.

4.1. Chave de Segurança

A chave tem 128 bits (16 bytes) e tem de ser rigorosamente igual no código do Emissor e do Recetor.

```

const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

```

4.2. Funções de Cifra e Decifra

Estas funções utilizam o contexto da biblioteca mbedts para transformar a estrutura de dados num buffer encriptado e vice-versa.

Função para Encriptar (No Emissor)

```

// Entrada: Struct organizada | Saída: Buffer de 16 bytes ilegíveis
void encryptPacket(const ResQSensePacket& inputPacket, uint8_t* outputBuffer) {
    mbedts_aes_context aes;

```

```

mbedtls_aes_init(&aes);
mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);

// O cast (unsigned char*) trata a struct como um array de bytes crus
mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)&inputPacket,
outputBuffer);

mbedtls_aes_free(&aes);
}

```

Função para Desencriptar (No Recetor)

// Entrada: Buffer encriptado | Saída: Struct organizada

```

void decryptPacket(const uint8_t* inputBuffer, ResQSensePacket* outputPacket) {
    mbedtls_aes_context aes;

    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, AES_KEY, 128);

    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, inputBuffer, (unsigned char*)outputPacket);

    mbedtls_aes_free(&aes);
}

```

6. Código do Emissor atualizado (Colete)

O coleite agora escuta pacotes de configuração. Se receber um pacote destinado ao seu ID, ajusta o temporizador (interval e offset) e responde com um ACK.

```

#include <SPI.h>
#include <LoRa.h>
#include "mbedtls/aes.h"

// HARDWARE
#define SCK 5

```

```

#define MISO 19
#define MOSI 27
#define SS 18
#define RST 14
#define DIO0 26
#define BAND 868E6

// DEFINIÇÕES DE PACOTES
#define PKT_TYPE_CONFIG 0xC1 // Tablet -> Colete (Configuração)
#define PKT_TYPE_ACK 0xC2 // Colete -> Tablet (Confirmação)

// CHAVE AES-128 (Igual para todos)
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

// ESTRUTURA DE DADOS
typedef struct __attribute__((packed)) {
    int8_t vestID;
    uint8_t spo2_header; // Flags + SpO2
    uint16_t heartRate;
    int16_t temperature;
    int32_t latitude;
    int32_t longitude;
    int16_t altitude;
} ResQSensePacket;

// PACOTE DE CONFIGURAÇÃO (Tablet -> Colete)
typedef struct __attribute__((packed)) {
    uint8_t targetVestID;
    uint8_t packetType; // Deve ser 0xC1
    uint16_t interval_ms; // Novo intervalo de envio
    uint16_t slot_offset_ms; // Atraso inicial (Time Slot)
    uint32_t master_epoch; // Timestamp
    uint8_t padding[6]; // Zeros para encher 16 bytes
} ControlPacket;

// PACOTE DE ACK (Colete -> Tablet)
typedef struct __attribute__((packed)) {

```

```

    int8_t vestID;
    uint8_t packetType;    // Deve ser 0xC2
    uint8_t status;        // 1 = OK
    int8_t batteryLevel;   // Nível bateria
    uint8_t padding[12];   // Zeros para encher 16 bytes
} AckPacket;

// Buffers
uint8_t encryptedBuffer[16];
uint8_t receivedBuffer[16];

// Variáveis de Controlo de Tempo (Substitui o delay)
unsigned long lastSendTime = 0;
unsigned long sendInterval = 10000; // Padrão 10s (pode ser alterado pelo Tablet)
int myID = 1; // ID deste colete

// FUNÇÃO DE PACKING
ResQSensePacket createPacket(int id, bool alertFlag, int spo2, int hr, float temp, float lat, float lon, int alt) {
    ResQSensePacket packet;
    packet.vestID = (int8_t)id;
    uint8_t flagBit = alertFlag ? 1 : 0;
    packet.spo2_header = (flagBit << 7) | (spo2 & 0x7F);
    packet.heartRate = (uint16_t)(hr & 0x01FF);
    packet.temperature = (int16_t)(temp * 100.0f);
    packet.latitude = (int32_t)(lat * 10000000.0f);
    packet.longitude = (int32_t)(lon * 10000000.0f);
    packet.altitude = (int16_t)alt;
    return packet;
}

// FUNÇÃO DE ENCRIPTAÇÃO
void encryptPacket(const ResQSensePacket& inputPacket, uint8_t* outputBuffer) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);
    // Nota: O cast (const unsigned char*) funciona para qualquer struct de 16 bytes
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)&inputPacket,
outputBuffer);
    mbedtls_aes_free(&aes);
}

```



```

// DESENCRIPTAÇÃO
void decryptPacket(const uint8_t* inputBuffer, uint8_t* outputBuffer) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, AES_KEY, 128);
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, inputBuffer, outputBuffer);
    mbedtls_aes_free(&aes);
}

void setup() {
    Serial.begin(115200);
    SPI.begin(SCK, MISO, MOSI, SS);
    LoRa.setPins(SS, RST, DIO0);

    if (!LoRa.begin(BAND)) {
        Serial.println("Erro ao iniciar LoRa!");
        while (1);
    }
    Serial.println("ResQSense TX Iniciado (Modo TDMA).");
}

void loop() {
    // Receber configuração do tablet (RX)
    int packetSize = LoRa.parsePacket();
    if (packetSize == 16) {
        LoRa.readBytes(receivedBuffer, 16);

        uint8_t decryptedRaw[16];
        decryptPacket(receivedBuffer, decryptedRaw); // Descripta

        // Interpreta como pacote de controlo
        ControlPacket* ctrl = (ControlPacket*)decryptedRaw;

        // Verifica se é tipo configuração (0xC1) e é para mim?
        if (ctrl->packetType == PKT_TYPE_CONFIG && ctrl->targetVestID == myID) {
            Serial.println("[RX] Configuração recebida! Sincronizando...");

            // Atualiza intervalo
            sendInterval = ctrl->interval_ms;
        }
    }
}

```

```

// Subtrai o offset ao tempo atual para "enganar" o timer e forçar o envio no slot correto
lastSendTime = millis() - ctrl->slot_offset_ms;

// ENVIAR ACK
AckPacket ack;
ack.vestID = myID;
ack.packetType = PKT_TYPE_ACK; // 0xC2
ack.status = 1;    // Sucesso
ack.batteryLevel = 85; // por ex
memset(ack.padding, 0, 12);

// Reutiliza encryptPacket(fazendo cast da struct ACK para a struct de dados)
encryptPacket((ResQSensePacket&)ack, encryptedBuffer);

LoRa.beginPacket();
LoRa.write(encryptedBuffer, 16);
LoRa.endPacket();
Serial.println("[TX] ACK enviado.");
}
}

// 2. ENVIAR DADOS DE SENSORES (TX) - Baseado em tempo (TDMA)
if (millis() - lastSendTime >= sendInterval) {
    lastSendTime = millis(); // Reseta o timer

    // Simulação de valores
    bool panicBtn = false;
    int valSpO2 = 98;
    int valHR = 72;
    float valTemp = 36.65;
    float valLat = 40.64427;
    float valLon = -8.64554;
    int valAlt = 15;

    // Criar o pacote para enviar para o tablet
    ResQSensePacket packet = createPacket(myID, panicBtn, valSpO2, valHR, valTemp, valLat, valLon,
    valAlt);

    // Encriptar e enviar

```

```

    encryptPacket(packet, encryptedBuffer);

    LoRa.beginPacket();
    LoRa.write(encryptedBuffer, 16);
    LoRa.endPacket();

    Serial.print("Dados enviados. Prox envio em: ");
    Serial.print(sendInterval); Serial.println("ms");
}
}

```

7. Código do Recetor atualizado (Tablet / Dashboard)

O Tablet agora consegue distinguir se o pacote recebido é um dado de sensor (para mostrar no dashboard) ou um ACK de um colete. Adicionalmente, incluí uma função para enviar o comando de configuração.

```

#include <SPI.h>
#include <LoRa.h>
#include "mbedtls/aes.h"

#define SCK    5
#define MISO   19
#define MOSI   27
#define SS     18
#define RST    14
#define DIO0   26
#define BAND   868E6

#define PKT_TYPE_CONFIG 0xC1
#define PKT_TYPE_ACK    0xC2

const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,

```

```
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C  
};
```

```
// Estruturas
```

```
typedef struct __attribute__((packed)) {  
    int8_t vestID;  
    uint8_t spo2_header;  
    uint16_t heartRate;  
    int16_t temperature;  
    int32_t latitude;  
    int32_t longitude;  
    int16_t altitude;  
} ResQSensePacket;
```

```
typedef struct {  
    int vestID;  
    bool isAlerting;  
    int spo2;  
    int heartRate;  
    float temperature;  
    float latitude;  
    float longitude;  
    int altitude;  
} DashboardData;
```

```
// Estruturas de Controlo
```

```
typedef struct __attribute__((packed)) {  
    uint8_t targetVestID;  
    uint8_t packetType;  
    uint16_t interval_ms;  
    uint16_t slot_offset_ms;  
    uint32_t master_epoch;  
    uint8_t padding[6];  
} ControlPacket;
```

```
typedef struct __attribute__((packed)) {  
    int8_t vestID;  
    uint8_t packetType;  
    uint8_t status;  
    int8_t batteryLevel;
```

```

    uint8_t padding[12];
} AckPacket;

// DESENCRIPTAÇÃO
void decryptPacket(const uint8_t* inputBuffer, uint8_t* outputBuffer) {
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, AES_KEY, 128);
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, inputBuffer, outputBuffer);
    mbedtls_aes_free(&aes);
}

// UNPACKING
DashboardData unpackPacket(const ResQSensePacket& packet) {
    DashboardData data;
    data.vestID = packet.vestID;
    data.isAlerting = (packet.spo2_header >> 7) & 0x01;
    data.spo2 = packet.spo2_header & 0x7F;
    data.heartRate = packet.heartRate & 0x01FF;
    data.temperature = (float)packet.temperature / 100.0f;
    data.latitude = (float)packet.latitude / 10000000.0f;
    data.longitude = (float)packet.longitude / 10000000.0f;
    data.altitude = packet.altitude;
    return data;
}

// --- FUNÇÃO PARA ENVIAR CONFIGURAÇÃO (TABLET -> COLETE) ---
void sendConfig(uint8_t targetID, uint16_t interval, uint16_t offset) {
    ControlPacket cfg;
    cfg.targetVestID = targetID;
    cfg.packetType = PKT_TYPE_CONFIG; // 0xC1
    cfg.interval_ms = interval;
    cfg.slot_offset_ms = offset;
    cfg.master_epoch = millis();
    memset(cfg.padding, 0, 6);

    uint8_t encBuf[16];
    // Encripta o pacote de configuração
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);

```

```

    mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);
    mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)&cfg, encBuf);
    mbedtls_aes_free(&aes);

    LoRa.beginPacket();
    LoRa.write(encBuf, 16);
    LoRa.endPacket();
    Serial.print("CONFIG enviada para Colete ID: "); Serial.println(targetID);
}

void setup() {
    Serial.begin(115200);
    SPI.begin(SCK, MISO, MOSI, SS);
    LoRa.setPins(SS, RST, DIO0);
    if (!LoRa.begin(BAND)) {
        Serial.println("Erro LoRa RX");
        while (1);
    }
    Serial.println("ResQSense RX (Tablet) Iniciado...");
    Serial.println("Enviar 'C' no Serial para configurar o Colete ID 1"); // para testar
}

void loop() {
    // RECEÇÃO
    int packetSize = LoRa.parsePacket();
    if (packetSize == 16) {
        uint8_t rxBuffer[16];
        uint8_t decryptedRaw[16];

        LoRa.readBytes(rxBuffer, 16);
        decryptPacket(rxBuffer, decryptedRaw);

        // Verificar o tipo de pacote (Byte 1)
        if (decryptedRaw[1] == PKT_TYPE_ACK) {
            // ACK
            AckPacket* ack = (AckPacket*)decryptedRaw;
            Serial.print("[ACK] Recebido do Colete "); Serial.println(ack->vestID);
            Serial.print("    Status: "); Serial.print(ack->status);
            Serial.print(" | Bateria: "); Serial.print(ack->batteryLevel); Serial.println("%");
        }
    }
}

```

```

else {
    // Assume-se dados se não for ACK
    ResQSensePacket* pkt = (ResQSensePacket*)decryptedRaw;
    DashboardData data = unpackPacket(*pkt);

    Serial.print("[DADOS] ID: "); Serial.print(data.vestID);
    Serial.print(" | Alerta: "); Serial.print(data.isAlerting ? "SIM" : "Nao");
    Serial.print(" | HR: "); Serial.print(data.heartRate);
    Serial.print(" | Temp: "); Serial.println(data.temperature);
}
}

// SIMULAÇÃO DE ENVIO DE CONFIGURAÇÃO
if (Serial.available()) {
    char c = Serial.read();
    if (c == 'C' || c == 'c') {
        // Configura Colete 1: Enviar a cada 5000ms, Offset 0ms
        sendConfig(1, 5000, 0);
    }
}
}
}

```

8. Protocolo de Sincronização e Controlo (TDMA)

Para evitar colisões na rede LoRa e organizar o envio de dados dos múltiplos operacionais, foi implementado um protocolo de inicialização do tipo Master-Slave. O Tablet (Mestre) atribui janelas de tempo (Time Slots) a cada Colete (Escravo).

Estrutura dos Pacotes de Controlo

Ao contrário do pacote de dados (ResQSensePacket do [ponto 2](#)), que contém leituras de sensores, os pacotes de controlo servem para configuração da rede. Para manter a

compatibilidade com o motor de encriptação AES-128, todos os pacotes de controlo têm de ter estritamente 16 bytes.

a. Pacote de Configuração (Tablet → Colete)

Este pacote é enviado pelo Tablet para definir quando o colete deve começar a transmitir.

Byte	Variável	Tipo	Descrição
0	targetVestID	uint8_t	ID do colete de destino.
1	packetType	uint8_t	Identificador do tipo de mensagem (Ex: 0xC1).
2-3	interval_ms	uint16_t	Novo intervalo de envio em ms (Ex: 10000).
4-5	slot_offset_ms	uint16_t	Tempo de espera ("atraso") antes do 1º envio.
6-9	master_epoch	uint32_t	Timestamp do Tablet para registo (por ex).
10-15	padding	uint8_t[6]	Zeros para completar o bloco de 16 bytes.

b. Pacote de Confirmação/ACK (Colete → Tablet)

Resposta imediata do colete confirmando que recebeu a configuração e está pronto.

Byte	Variável	Tipo	Descrição
0	vestID	int8_t	ID do colete que está a responder.
1	packetType	uint8_t	Identificador de resposta (Ex: 0xC2).
2	status	uint8_t	Ex: Estado: 1 = Sucesso, 0 = Erro.
3	batteryLevel	int8_t	Nível da bateria (%).

Byte	Variável	Tipo	Descrição
4-15	padding	uint8_t[12]	Zeros para completar o bloco de 16 bytes.

Definições Comuns e Estruturas

Manter a diretiva `__attribute__((packed))` para não acontecer padding automático do compilador.

```
#include <SPI.h>
#include <LoRa.h>
#include "mbedtls/aes.h"

// DEFINIÇÕES DE PACOTES
#define PKT_TYPE_CONFIG 0xC1 // Tablet -> Colete (Configuração)
#define PKT_TYPE_ACK 0xC2 // Colete -> Tablet (Confirmação)

// CHAVE AES-128 (Igual para todos)
const unsigned char AES_KEY[16] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C
};

// ESTRUTURA DE DADOS
typedef struct __attribute__((packed)) {
    int8_t vestID;
    uint8_t spo2_header; // Flags + SpO2
    uint16_t heartRate;
    int16_t temperature;
    int32_t latitude;
    int32_t longitude;
    int16_t altitude;
} ResQSensePacket;

// PACOTE DE CONFIGURAÇÃO (Tablet -> Colete)
typedef struct __attribute__((packed)) {
```

```

uint8_t targetVestID; // Para quem é este pacote?
uint8_t packetType; // Deve ser 0xC1
uint16_t interval_ms; // Novo intervalo de envio
uint16_t slot_offset_ms; // Atraso inicial (Time Slot)
uint32_t master_epoch; // Timestamp
uint8_t padding[6]; // Zeros para encher 16 bytes
} ControlPacket;

// PACOTE DE ACK (Colete -> Tablet)
typedef struct __attribute__((packed)) {
    int8_t vestID; // Quem está a responder
    uint8_t packetType; // Deve ser 0xC2
    uint8_t status; // 1 = OK
    int8_t batteryLevel; // Nível bateria
    uint8_t padding[12]; // Zeros para encher 16 bytes
} AckPacket;

```

Lógica de Receção no Firmware

O fluxo de funcionamento do colete teve de ser atualizado para processar estes novos pacotes:

1. **Descriptação:** O colete recebe 16 bytes e descripta usando a chave partilhada AES_KEY. Como o AES-128 opera em blocos fixos de 16 bytes, podemos usar a mesma função de encriptação para qualquer uma das 3 structs, desde que façamos o cast correto (*unsigned char**) no código do tablet.
2. **Identificação:** Verifica o byte packetType.
 - Se for Dados, ignora (o colete não processa dados de outros coletes).
 - Se for Configuração (0xC1) e *targetVestID == myID*, processa.
3. **Sincronização:**
 - Atualiza a variável *sendInterval* com *packet.interval_ms*.
 - Ajusta o temporizador interno (*lastSendTime*) subtraindo o *slot_offset_ms* ao tempo atual. Isto força o próximo envio a acontecer no momento exato definido pelo Tablet, sem bloquear o processador com *delay()*.

Mudanças no código do emissor atualizado

- **(Slot Offset):** A linha *lastSendTime = millis() - ctrl->slot_offset_ms*; é o "coração" da sincronização. Ao subtrair o offset (atraso) ao tempo atual, "enganamos" o

microcontrolador fazendo-o pensar que o último envio ocorreu no passado exato necessário para que o próximo envio ($\text{lastSendTime} + \text{interval}$) coincida com o Time Slot atribuído pelo Tablet. Isto evita o uso de `delay()` bloqueante.

- **Modo Fallback:** O código inicia com `sendInterval = 10000` (10 segundos) por defeito. Isto garante que, se o Colete nunca receber um comando de configuração do Tablet (por estar fora de alcance inicial), ele continua a enviar dados periodicamente, embora sem garantia de evitar colisões.
- **Filtragem de IDs:** A verificação `if (ctrl->targetVestID == myID)` impede que o colete re programe o seu horário ao ouvir uma configuração destinada a outro colete vizinho.

Mudanças no [código do recetor atualizado](#)

- **Distinção de Pacotes:** Como o LoRa apenas entrega um array de bytes crus, o recetor tem de "adivinhar" o que recebeu.
 - A lógica verifica o byte de índice 1 (`packetType`). Nos pacotes de controlo, este byte é 0xC1 ou 0xC2. Nos pacotes de dados de sensores (`ResQSensePacket`), o byte 1 corresponde a parte das Flags e SpO2.

Nota de Risco: No futuro, pode-se mover o `packetType` para o byte 0 ou usar um identificador mais robusto, para não perder dados de SpO2.

- **Time Slot:** No colete, a linha `lastSendTime = millis() - ctrl->slot_offset_ms;` é crítica. Se o Tablet diz "O intervalo é 10s e o teu atraso é 2s", ao receber o comando no instante T, o colete finge que enviou dados em $T - 2s$. Assim, o próximo envio será em $(T - 2s) + 10s = T + 8s$, garantindo que ele não colide com o colete do Slot 0 (que enviaria em $T + 10s$).

9. Conformidade Regulamentar (Banda ISM)

O sistema ResQSense opera na banda não licenciada ISM (Industrial, Scientific, and Medical) de 868 MHz, em conformidade com a norma europeia ETSI EN 300 220.

Para garantir a legalidade e a coexistência com outros dispositivos na mesma banda, existem as seguintes restrições:

- Duty Cycle de 1%: Cada dispositivo só pode transmitir 1% do tempo por hora (36 s).
- Potência de Transmissão: (Limite Legal) A potência máxima irradiada permitida é de 14 dBm (25 mW). Nota Técnica: Temos de garantir (via software `LoRa.setTxPower(14)`) ou documentar que o ganho da antena somado à potência de transmissão não excede os 14 dBm.
- Gestão de Colisões: A implementação do protocolo TDMA (Time Division Multiple Access) assegura que múltiplos coletes não transmitem simultaneamente, minimizando a poluição espectral e a perda de pacotes por colisão.

Análise de Tempo de Antena (Time on Air - ToA)

Para o pacote padrão de 16 bytes, utilizando SF7:

1. ToA Calculado: ~51,5 ms por pacote.
2. Intervalo de Envio: 10.000 ms (10 segundos).

Cálculo de Ocupação (Duty Cycle)

Numa janela de 1 hora (3600 segundos), a ocupação espectral teórica é:

$$Duty\ Cycle = \frac{51,5\ ms}{10000\ ms} \approx 0,51\%$$

Conclusão de Legalidade: O sistema ocupa apenas 0,51% do tempo, estando bem abaixo do limite legal de 1%.

- Margem de Segurança: A "sobra" de ~0,49% permite o envio de pacotes de controlo (ACKs e Syncs) sem violar a lei.
- Restrição de SF: O uso de Spreading Factors superiores a SF8 (SF9-SF12) com o intervalo atual de 10s é proibido, pois excederia o limite de 1%. Se for necessário maior alcance, o intervalo de envio deverá ser aumentado proporcionalmente.
- Potência: A potência de transmissão está limitada por software a 14 dBm (25 mW).

10. Notas finais a ter em conta para os proximos passos

Testar com duas ESP32 os códigos atualizados para testar e confirmar os valores de Time on Air e SpreadFactor e confirmar que o recetor recebe tudo sem perdas mesmo em situações adversas, de forma a avançar para a leitura dos valores nos sensores e passagem para o dashboard. Testes de Campo: Validar o alcance real e a robustez do TDMA com múltiplos emissores simultâneos.

Testar Colisões: Realizar o teste com pelo menos 3 dispositivos para validar se o slot_offset_ms realmente evita a sobreposição em frequências LoRa congestionadas.

Para verificar a encriptação "no ar", usar o wireshark de forma a captar os pacotes LoRa wireless e confirmar se esta tudo a funcionar como está projetado. (Ver disponibilidade de testar na sala de Redes já com tudo funcional).

Uniformizar o Header(Reservar o Byte 0 para o packetType em todas as structs)

Monitorização de Bateria: No AckPacket, já está previsto o nível de bateria; será útil adicionar esse campo também no pacote de dados regular (ResQSensePacket) para monitorização contínua no Dashboard

Dashboard: Finalizar a integração dos alertas (Flags) na interface do Tablet.

11. Bibliografia

Encriptação:

<https://github.com/alvingigo/AES-128/blob/main/aes128.cpp> (Chave de segurança)

<https://mbed-tls.readthedocs.io/en/latest/kb/how-to/encrypt-with-aes-cbc/> (mbedtls)

TDMA:

12. Historico de versões

Versão	Data	Descrição das Alterações
1.0	25/01/2026	Versão inicial da documentação
1.1	28/01/2026	Otimização do Payload: Reestruturação do formato da mensagem para incluir flags de temperatura crítica (alta/baixa) e Man Down através da reutilização de bits que eram de padding, mantendo o pacote em 16 bytes. Atualização das notas finas.
1.2	30/01/2026	Implementação do Protocolo de Sincronização (TDMA): Definição de novas estruturas de controlo (<i>ConfigPacket</i> e <i>AckPacket</i>) com alinhamento de 16 bytes para compatibilidade AES. Introdução da lógica de Time Slots para gestão de múltiplos dispositivos na rede LoRa. Atualização dos codigos para a comunicação colete ↔ tablet
1.3	8/02/2026	Adição da introdução ao documento. Reestruturação do formato da mensagem para incluir tambem flags de bpm crítica (alta/baixa), bateria baixa e SOS. Conformidade Regulamentar (Banda ISM): detalhe das regras e valores legais da banda free de Europa (868 MHz).